

Modélisation avancée des Systèmes d'Information

06 - Strategy

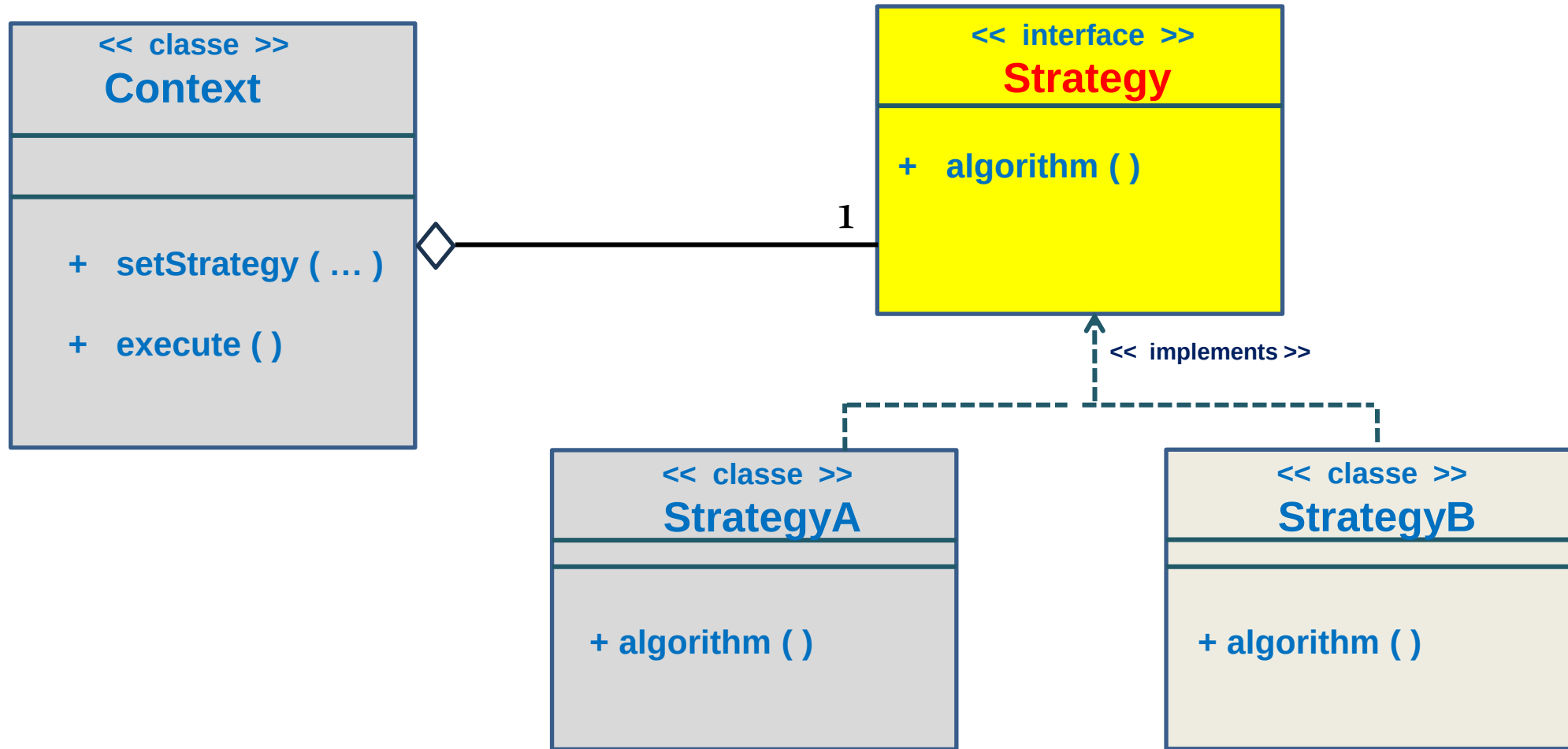
Mohamed ZAYANI

2025/2026

Définition de Strategy

- Le patron de conception Strategy est **un patron comportemental** qui permet de définir une famille d'algorithmes, de les encapsuler dans des classes distinctes et de les rendre interchangeables.
- **Objectif principal:**
 - ❖ **Modifier dynamiquement le comportement d'un objet sans altérer son code.**
- **Problématique:**
 - Comment permettre à un objet de choisir dynamiquement parmi plusieurs algorithmes sans multiplier les conditions (if-else ou switch) ?
 - Comment respecter le principe Open/Closed (ouvert à l'extension, fermé à la modification) sans modifier la classe principale ?.
- **Solution:**
 - Définir une interface commune pour tous les algorithmes.
 - Créer des implémentations concrètes de cette interface pour chaque stratégie.
 - Permettre à la classe cliente de **choisir dynamiquement** la stratégie à appliquer.
 - **L'ajout de nouveaux comportements se fait sans modifier le code existant**

Modélisation UML



Implémentation JAVA

```
public interface StrategieTri
{
    void trier(int[] tableau);
}
```

```
public class TriInsertion implements StrategieTri
{
    @Override
    public void trier(int[] tableau)
    {
        for (int i = 1; i < tableau.length; i++)
        {
            int key = tableau[i];
            int j = i - 1;
            while (j >= 0 && tableau[j] > key)
            {
                tableau[j + 1] = tableau[j];
                j--;
            }
            tableau[j + 1] = key;
        }
    }
}
```

```
public class TriSelection implements StrategieTri
{
    @Override
    public void trier(int[] tableau)
    {
        int n = tableau.length;
        for (int i = 0; i < n - 1; i++)
        {
            int minIndex = i;
            for (int j = i + 1; j < n; j++)
            {
                if (tableau[j] < tableau[minIndex])
                {
                    minIndex = j;
                }
            }
            int temp = tableau[minIndex];
            tableau[minIndex] = tableau[i];
            tableau[i] = temp;
        }
    }
}
```

Implémentation JAVA

```
public class Contexte {
    //Attribut d'association
    private StrategieTri strategie;
    // Setter pour changer la stratégie dynamiquement
    public void setStrategie(StrategieTri strategie)
    {
        this.strategie = strategie;
    }

    // Exécution de la stratégie de tri
    public void trierTableau(int [] tableau) {
        if (strategie == null) {
            throw new IllegalStateException(
                "Aucune stratégie de tri n'a été définie.");
        }
        strategie.trier(tableau);
    }
}
```

Résultat:

```
Résultat : [1, 2, 5, 5, 6, 9]
Résultat : [1, 2, 5, 5, 6, 9]
```

```
import java.util.Arrays;
public class Client {

    public static void main(String[] args) {
        // Création du contexte
        Contexte contexte = new Contexte();

        int[] tableau = {5, 2, 9, 1, 5, 6};
        // Utilisation du tri par sélection
        contexte.setStrategie(new TriSelection());
        contexte.trierTableau(tableau);
        System.out.println("Résultat : " +
            Arrays.toString(tableau));

        // Réinitialisation du tableau
        tableau = new int[]{5, 2, 9, 1, 5, 6};

        // Utilisation du tri par insertion
        contexte.setStrategie(new TriInsertion());
        contexte.trierTableau(tableau);
        System.out.println("Résultat : " +
            Arrays.toString(tableau));
    }
}
```

Cas d'utilisation du Patron Strategy

- **Algorithmes de tri** ou de compression interchangeables.
- **Moteurs de paiement** qui changent de stratégie selon le moyen de paiement (carte, PayPal, crypto).
- **Jeux vidéo** : Stratégies d'attaque différentes selon les niveaux.
- **Systèmes de navigation** : Choix de l'itinéraire (rapide, économique, panoramique).